LSIS-AFS Reference Implementation - Competition Spec

Overview

Software reference implementation for LunaNet Signal-In-Space Augmented Forward Signal (LSIS-AFS) Volume A. This focuses on the digital signal processing, encoding/decoding, and message generation/parsing components that can be developed and tested in software.

Reference Documentation

LSIS-AFS Volume A Specification

Competition Scope

In Scope (Software Implementation)

- Spreading code generation (Gold, Weil, Legendre sequences)
- Digital baseband signal generation (I/Q samples)
- Message encoding/decoding (BCH, LDPC, CRC)
- Frame structure and message parsing
- Time of transmission calculation
- SISE calculation algorithms
- Test vector generation and validation

Out of Scope (RF Hardware)

- RF carrier generation and modulation
- Antenna design and polarization
- Power amplifiers and transmitters
- Receiver front-end hardware
- Phase noise and RF impairments

Key Deliverables

The implementation is structured around **8 gateways** - incremental milestones that each deliver a complete, testable capability:

Gateway Principle: Each gateway represents a functional building block with clear success criteria. This approach enables: - Incremental validation and early error detection - Parallel development of independent components
- Measurable progress tracking over 6 months - Partial credit for completed gateways - Natural integration points between components

1. **Encoder**: Generate AFS signal baseband (I/Q samples) from navigation data
2. **Decoder**: Extract navigation data from AFS baseband samples
3. **Message Builder**: Create navigation messages from ephemeris/clock data
4. **Message Parser**: Extract fields from received navigation messages
5. **Test Suite**: Validate implementation against specification
6. **Documentation**: API documentation and usage examples

Technical Specifications Summary

Code Parameters

Parameter	AFS-I	AFS-Q
Primary Code	Gold, 2046 chips	Weil, 10230 chips

Parameter	AFS-I	AFS-Q
Chip Rate	1.023 Mchip/s	5.115 Mchip/s
Code Period	2 ms	2 ms
Secondary Code	N/A	4 chips, 8 ms
Tertiary Code	N/A	1500 chips, 12 s
Symbol Rate	500 symbols/s	Pilot (no data)

Frame Structure

Component	Bits	Encoding	Symbols
Sync Pattern	-	None	68
Subframe 1	9	BCH(51,8)	52
Subframe 2	1200	LDPC(1/2)	2400
Subframe 3	870	LDPC(1/2)	1740
Subframe 4	870	LDPC(1/2)	1740
Total	-	-	**6000**

Time Parameters

- **Frame Duration**: 12 seconds
- **Block Interval**: 1200 seconds (20 minutes)
- **TOI Range**: 0-99 (100 frames per block interval)
- **ITOW Range**: 0-503 (504 block intervals per week)
- **Week Number**: 0-8191 (13 bits, ~157 years)

Implementation Architecture

Suggested Module Structure

```
lsis-afs/
  codes/
      gold            # Gold code generator
      weil            # Weil code generator
      legendre        # Legendre sequence generator
      tiered          # Tiered code assembly
  encoding/
      bch             # BCH(51,8) encoder/decoder
      ldpc            # LDPC encoder/decoder
      crc             # CRC-24 generator/validator
      interleaver     # Block interleaver
  messages/
      subframe1       # SB1 builder/parser
      subframe2       # SB2 builder/parser
      subframe3       # SB3 builder/parser
      subframe4       # SB4 builder/parser
      frame           # Frame assembly/disassembly
  signal/
      modulator       # Baseband I/Q generation
      demodulator     # Symbol extraction
      sync            # Frame synchronization
  utils/
      sise            # SISE calculations
```

```
    time             # ToT calculations
    codes_db         # Code assignment database
tests/
    test_codes
    test_encoding
    test_messages
    test_integration
data/
    ldpc_matrices/   # LDPC matrix files from Annex1
    reference_codes/ # Reference codes from Annex3
    test_vectors/    # Generated test vectors
```

Key Algorithms to Implement

1. **Gold Code Generation**: 11-stage LFSR with feedback taps
2. **Legendre Sequence**: Quadratic residue calculation mod prime
3. **Weil Code Construction**: Legendre sequence with insertion
4. **BCH Encoding**: Linear shift register with generator polynomial
5. **LDPC Encoding**: Sparse matrix multiplication in GF(2)
6. **LDPC Decoding**: Belief propagation (sum-product algorithm)
7. **CRC Calculation**: Polynomial division in GF(2)
8. **Frame Sync**: Cross-correlation with sync pattern

Requirements

Phase 1: Core Spreading Code Generation (HIGH PRIORITY)

1.1 Gold Code Generator (AFS-I Primary)

Implement 11-stage LFSR for Gold code generation
Generate 2046-chip sequences (short-cycled from 2047)
Support PRN 1-210 with initialization vectors from Appendix C
Output codes in binary and hexadecimal formats
Validate against reference codes in Annex3

1.2 Weil Code Generator (AFS-Q Primary)

Implement Legendre sequence generator (length 10223)
Implement Weil code construction from Legendre sequence
Generate 10230-chip sequences for PRN 1-210
Apply insertion index per Appendix D tables
Validate against reference codes in Annex3

1.3 Secondary Code Generator (AFS-Q)

Implement 4 secondary code sequences (S0, S1, S2, S3)
Map codes per Table 10: S0=1110, S1=0111, S2=1011, S3=1101

1.4 Tertiary Weil Code Generator (AFS-Q)

Implement Legendre sequence generator (length 1499)
Generate 1500-chip tertiary Weil codes for PRN 1-210
Apply Weil index k per Appendix E tables
Validate against reference codes in Annex3

1.5 Tiered Code Assembly

Implement modulo-2 addition of primary + secondary codes
Implement modulo-2 addition with tertiary code
Ensure coherent generation (no chip slips)
Generate full 12-second composite code sequence

Phase 2: Digital Signal Generation (HIGH PRIORITY)

2.1 Baseband I/Q Sample Generation

Generate AFS-I baseband: data symbols Gold code at 1.023 Mchip/s
Generate AFS-Q baseband: tiered code at 5.115 Mchip/s
Implement BPSK mapping: logic 1 → -1.0, logic 0 → +1.0
Generate complex I/Q samples: I(t) + jQ(t)
Support configurable sample rate (e.g., 10.23 MHz, 20.46 MHz)

2.2 Code Synchronization

Align data symbols with primary code boundaries (LSIS-160)
Align secondary code with primary code boundaries (LSIS-170)
Align tertiary code with secondary code boundaries (LSIS-171)
Synchronize tertiary code start with frame start (LSIS-220)

2.3 Signal File I/O

Export I/Q samples to binary format (int16, float32)
Export I/Q samples to CSV for analysis
Import I/Q samples for decoder testing
Support chunked processing for long signals

Phase 3: Message Encoding (HIGH PRIORITY)

3.1 BCH(51,8) Encoder for Subframe 1

Implement 8-stage LFSR with generator polynomial 763 (octal)
Encode 8 LSBs of SB1 to 51 symbols
Handle MSB modulo-2 addition and prepending
Generate 52-symbol encoded output
Create test vectors for validation

3.2 LDPC Encoder

Load LDPC matrices from Annex1 CSV files
Parse submatrices A, B, C, D, B [1] for SB2 (1200 bits)
Parse submatrices A, B, C, D, B [1] for SB3/SB4 (870 bits)
Implement rate 1/2 LDPC encoding algorithm
Handle puncturing: first z bits of systematic portion
Handle filler bits for SB3/SB4 (10 zero bits)
Generate parity bits p1 and p2
Output encoded symbols (s*; p*)

3.3 CRC-24 Generator

Implement CRC-24 with generator polynomial from LSIS-FID0-467
Apply to all SB2, SB3, SB4 data including spare bits
Compute 24 parity bits using polynomial division
Append CRC to data fields

3.4 Block Interleaver

Implement 60×98 block interleaver for 5880 symbols
Write symbols row-wise (left to right, top to bottom)
Read symbols column-wise (top to bottom, left to right)
Validate interleaving pattern

Phase 4: Frame Structure & Message Building (HIGH PRIORITY)

4.1 Synchronization Pattern

Generate 68-symbol sync pattern: 0xCC63F74536F49E04A
Prepend to every frame (uncoded)

4.2 Subframe 1 Builder

Pack FID (2 bits) and TOI (7 bits) into 9-bit field
Apply BCH(51,8) encoding
Output 52 encoded symbols

4.3 Subframe 2 Builder (Clock & Ephemeris)

Define data structure for 1176 bits + 24-bit CRC
Pack Time of Transmission fields (WN, ITOW, TOI)
Pack Clock and Ephemeris Data (CED) fields
Pack Health and Safety status
Pack Time Conversions data
Add spare bits (fill with alternating 0/1 starting with 0)
Compute and append CRC-24
Apply LDPC encoding → 2400 symbols

4.4 Subframe 3 Builder (Variable Data)

Define data structure for 846 bits + 24-bit CRC
Pack SB3 type field (4 or 6 bits)
Support Multiple Orbit Almanac message format
Support Coordinate Frame Conversions format
Add filler bits and spare bits
Compute and append CRC-24
Apply LDPC encoding → 1740 symbols

4.5 Subframe 4 Builder (Network Access)

Define data structure for 846 bits + 24-bit CRC
Pack SB4 type field (4 or 6 bits)
Support LunaNet Network Access Information format
Support optional message formats
Add filler bits and spare bits
Compute and append CRC-24
Apply LDPC encoding → 1740 symbols

4.6 Frame Assembly

Concatenate: SP + SB1 + SB2 + SB3 + SB4
Apply interleaving to SB2 + SB3 + SB4 symbols
Output complete 12-second frame structure
Generate symbol stream for modulation

Phase 5: Message Decoding (MEDIUM PRIORITY)

5.1 Frame Synchronization

Implement sync pattern detection (correlation)
Detect frame boundaries in received symbol stream
Handle timing offsets and Doppler

5.2 BCH(51,8) Decoder

Implement soft-decision correlation decoder
Test all 256 codeword hypotheses
Select maximum correlation codeword
Extract 9-bit SB1 data (FID + TOI)

5.3 LDPC Decoder

Implement belief propagation algorithm
Restore punctured bits as erasures (LLR = 0)
Decode SB2 (2400 symbols → 1200 bits)
Decode SB3/SB4 (1740 symbols → 870 bits)
Remove filler bits from decoded data

5.4 CRC Validation

Compute CRC-24 on received data
Compare with received CRC
Flag corrupted frames

5.5 Block Deinterleaver

Implement inverse 60×98 deinterleaving
Write symbols column-wise
Read symbols row-wise

Phase 6: Message Parsing (MEDIUM PRIORITY)

6.1 Subframe 1 Parser

Extract FID (2 MSBs)
Extract TOI (7 LSBs)
Validate FID value (0-3)
Validate TOI value (0-99)

6.2 Subframe 2 Parser

Extract Week Number (WN) - 13 bits
Extract Interval Time of Week (ITOW) - 9 bits
Extract Clock and Ephemeris Data fields
Extract Health and Safety status
Extract Time Conversions data
Parse spare bits

6.3 Subframe 3 Parser

Extract SB3 type field
Route to appropriate message parser based on type
Parse Multiple Orbit Almanac data

Parse Coordinate Frame Conversions

6.4 Subframe 4 Parser

Extract SB4 type field
Route to appropriate message parser based on type
Parse LunaNet Network Access Information
Parse optional messages

6.5 Time Reconstruction

Implement ToT calculation: t = WN×Tw + ITOW×Tbi + TOI×Tf
Convert to seconds since LRT epoch
Add code phase tracking for sub-frame resolution

Phase 7: Utilities & Algorithms (MEDIUM PRIORITY)

7.1 SISE Calculation

Implement SISE position error calculation (Equation 1)
Implement SISE velocity error calculation (Equation 2)
Calculate 95th percentile over 24-hour window
Validate against 40m position / 1 cm/s velocity limits

7.2 Code Assignment Management

Load code assignments from Table F-1 (Appendix F)
Map LNSP node ID to PRN codes (I, Q primary, secondary, tertiary)
Support interim test codes (Table 11, PRN 1-12)
Handle tertiary phase offsets

7.3 Bit/Byte Ordering

Implement MSB-first bit ordering
Implement MSB-first byte ordering
Validate against Figure 6 specification

Phase 8: Testing & Validation (HIGH PRIORITY)

8.1 Unit Tests

Test Gold code generation for all 210 PRNs
Test Weil code generation for all 210 PRNs
Test BCH encoder/decoder with known vectors
Test LDPC encoder/decoder with known vectors
Test CRC-24 computation
Test interleaver/deinterleaver

8.2 Integration Tests

Test complete encoding chain: data → frame → symbols
Test complete decoding chain: symbols → frame → data
Test round-trip: encode → decode → verify
Test with multiple PRN codes
Test with various message types

8.3 Reference Test Vectors

Generate test vectors for spreading codes
Generate test vectors for BCH encoding
Generate test vectors for LDPC encoding
Generate test vectors for complete frames
Generate I/Q sample files for signal testing

8.4 Performance Testing

Measure encoding throughput
Measure decoding throughput
Measure BER vs SNR performance
Validate LDPC decoder convergence

8.5 Compliance Validation

Verify code lengths match specification
Verify frame timing (12-second frames)
Verify symbol rates (500 symbols/s for AFS-I)
Verify bit allocations for all subframes
Cross-check against LSIS requirement IDs

Development Priorities

Month 1: Design & Architecture

1. System architecture design
2. Technology stack selection
3. Development environment setup
4. Testing framework establishment
5. Interface definitions

Month 2: Foundation

1. Spreading code generators (Gold, Weil, Legendre)
2. BCH encoder/decoder
3. Basic I/Q signal generation

Month 3: Encoding & Messaging

1. LDPC encoder implementation
2. CRC-24 implementation
3. Interleaver implementation
4. Frame structure builders

Month 4: Signal Generation

1. Complete signal generation pipeline
2. Code synchronization
3. I/Q sample export

Month 5: Decoding & Parsing

1. LDPC decoder implementation
2. Frame synchronization
3. Message parsers

4. Time reconstruction

Month 6: Testing & Polish

1. Comprehensive test suite
2. Test vector generation
3. Performance optimization
4. Documentation

Success Criteria

Generate valid spreading codes matching Annex3 references
Encode/decode frames with BER < 10 at SNR > 0 dB
Process 12-second frames in < 1 second (real-time capable)
Pass all compliance tests against LSIS specifications
Generate complete test vector suite
Provide working examples for all 12 interim test codes